4. Functions, Multiple Returns, and Error Basics

04. Functions, Multiple Returns, and Error Basics

Phase: Phase 1 — CS fundamentals through Go
Estimated time: 1 day
Slug: `04-functions-errors`

Learning objectives

- Define functions with parameters and multiple return values
- Use the error interface as Go's primary failure signal
- Name return values when it improves clarity
- Avoid panic for ordinary errors

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Multiple returns and explicit `error` values make failure paths visible. This is the heartbeat of reliable Go services — no hidden exceptions.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- `(T, error)` is the dominant result pattern; check `err != nil` immediately.
- Errors are values implementing `Error() string` — often created with `errors.New` / `fmt.Errorf`.
- `panic` is for programming bugs, not expected failures like bad user input.
- Named results document intent; avoid them when they encourage shadowing.
- Never ignore errors with _ unless you can justify it in a comment.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-functions, && cd /tmp/gw-functions,
go mod init example.com/scratch
```

Then create `main.go`:

```go
package main

import (
    "errors"
    "fmt"
)

func Div(a, b int) (int, error) {
    if b == 0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}

func main() {
    v, err := Div(10, 2)
    fmt.Println(v, err)
    _, err = Div(1, 0)
    fmt.Println(err)
}
```

Run it:

```
go run .
```

Expected output:

```
5 <nil>
division by zero
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Write Div; call with success and failure.
2. Parse with strconv.Atoi and handle the error.
3. Return fmt.Errorf("parse %q: %w", s, err) once you know %w (preview).
4. Add a unit test style check in main: fail loudly if err is wrong.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

Functions

```go
func Add(a, b int) int {
    return a + b
}

func Div(a, b int) (int, error) {
```

```
    if b == 0 {
        return 0, errors.New("division by zero")
    }
    return a / b, nil
}
```

Idiom: last return value is often `error`. Callers check immediately:

```
v, err := Div(x, y)
if err != nil {
    return err
}
```

Why not exceptions?

Go's explicit errors make control flow visible. You see every failure path. `panic` exists for truly unrecoverable programmer mistakes — not for "file not found".

Named returns

```
func Split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return
}
```

Use sparingly; great for docs, easy to misuse with shadowing.

Failure modes to watch while coding

- Ignoring `err` with _ habitually
- Using panic like Python raise for normal failures
- Returning error strings inconsistently (prefer `fmt.Errorf` / `errors.New`)

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- Python raises; Go returns error values you must handle.
- Do not use panic/recover like try/except for ordinary control flow.
- Ignoring err is like `except Exception: pass` — worse in production.
- No finally — use `defer` for cleanup (later lessons deepen this).

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

Error messages should help operators: include the input or resource name, not only "failed".

Practice checklist

Before the exercise, confirm you can:

1. Explain “Functions, Multiple Returns, and Error Basics” aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson’s rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Ignoring `err` with _ habitually
- Using panic like Python raise for normal failures
- Returning error strings inconsistently (prefer `fmt.Errorf` / `errors.New`)

Exercises

Work in `exercises/04-functions-errors/`. Read that folder’s `README.md` before you code.

What you will build

Implement ParsePositive(s): Atoi, then reject n<=0 with a clear error.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/04-functions-errors/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Why is `(T, error)` preferred over exceptions here?
2. What should you do immediately after calling a function that returns error?
3. When is panic appropriate?

Further reading

- https://go.dev/blog/error-handling-and-go
- https://go.dev/tour/basics